

BACKEND API

Documentacao Tecnica do Projeto Escolar

API REST em ASP.NET Core 10 para autenticao JWT, autorizacao por perfil e CRUD de alunos, professores, usuarios e diretoria.

Este documento descreve arquitetura, camadas, comunicacao, regras de negocio, migrations, validacoes, testes, logging e modelo de dados.

Projeto: **form_API**

Repositorio: **SPA-Vue / Backend_API**

Gerado em: **22/05/2026**

Indice

- Visao geral e tecnologias
- Tipo de arquitetura e responsabilidades das camadas
- Diagramas de fluxo, classes, comunicacao e modelo entidade-relacional
- Regras de negocio e seguranca
- Migrations, FluentValidation, logging e Swagger
- CRUDs, endpoints, testes e execucao local

Visao Geral

O backend expõe uma API REST para uma aplicacao escolar. Ele gerencia alunos, professores, integrantes da diretoria, usuarios autenticaveis e perfis de acesso.

Framework

ASP.NET Core 10 Web API.

Persistencia

Entity Framework Core com SQL Server e SQLite para desenvolvimento/testes.

Seguranca

JWT Bearer, roles e hash de senha PBKDF2-SHA256.

Documentacao

Swagger/OpenAPI com suporte a Bearer Token.

Validacao

FluentValidation registrado no pipeline MVC.

Observabilidade

Logs diarios em `Backend_API/logs`.

Tipo de Arquitetura

O projeto usa arquitetura em camadas. A camada HTTP conversa com services, os services executam regras de aplicacao e delegam persistencia ao repository ou ao DataContext. ViewModels isolam contratos externos das entidades EF Core.

API

Controllers, middlewares, autenticacao, autorizacao, Swagger e respostas HTTP.

Aplicacao

Services com regras de negocio, orquestracao e retorno de ViewModels.

Infraestrutura

Repository, DataContext, migrations, hashing, logging e acesso ao banco.

Dominio/Dados

Models, relacionamentos EF Core, seeds e constraints.

Diagrama de Fluxo da Requisicao



O retorno percorre o caminho inverso: banco, EF Core, repository, service, ViewModel, controller e resposta HTTP. O middleware de logs registra inicio, fim, status e tempo de execucao.

Diagrama de Comunicacao Entre Camadas

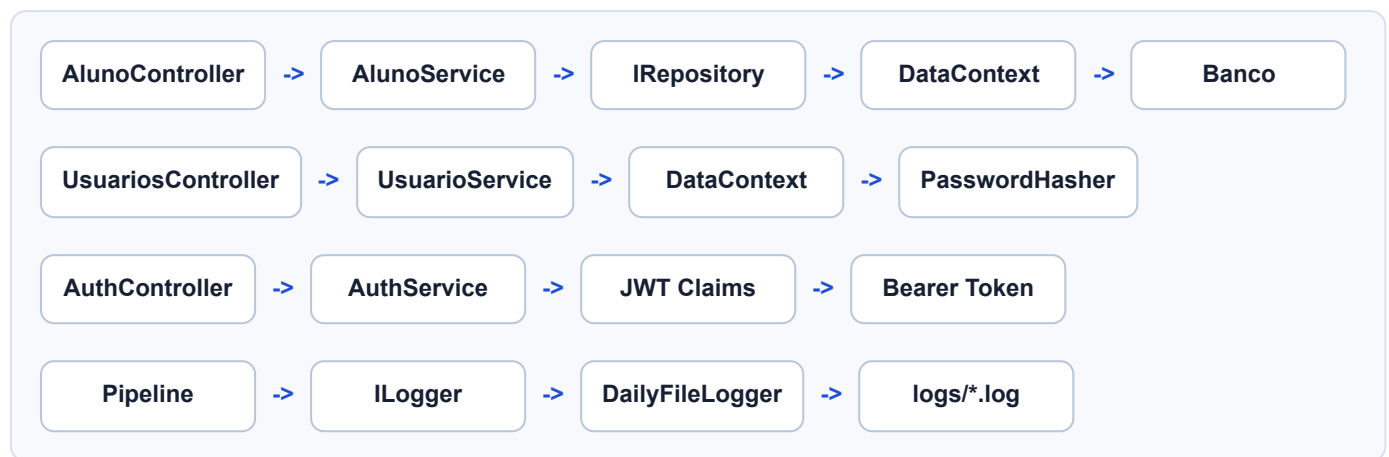


Diagrama de Classes



Relacionamentos principais: Perfil 1:N Usuario , Usuario 1:N Aluno , Usuario 1:N Professor , Usuario 1:N Diretoria e Professor 1:N Aluno .

Modelo Entidade-Relacional



ALUNO

- Id PK
- Nome
- Sobrenome
- DataNasc
- ProfessorId FK
- IdUsuario FK opcional

DIRETORIA

- Id PK
- Nome
- IdUsuario FK opcional

Ao excluir usuario, os vinculos opcionais em aluno, professor e diretoria sao configurados como nulos. O email de usuario possui indice unico.

Regras de Negocio

- Todo endpoint de negocio exige JWT valido.
- **Administrador** possui acesso total a CRUDs, usuarios e perfis.
- **Contribuinte** cria e atualiza alunos, professores e diretoria, mas nao exclui.
- **Leitor** consulta endpoints autenticados de leitura.
- Usuarios novos recebem senha padrao `Senha@252525`.
- Ao usar a senha padrao, a aplicacao sinaliza troca obrigatoria antes de liberar o fluxo normal.
- Senha e armazenada apenas como hash PBKDF2-SHA256.
- Aluno precisa de professor valido.
- Aluno, professor e diretoria podem ou nao ter usuario vinculado.
- Data de nascimento deve estar no formato `dd/MM/yyyy`.

Autenticacao e Autorizacao

O endpoint `POST /api/Auth/login` valida email e senha, gera JWT com claims de usuario e perfil, e retorna a flag `deveAlterarSenhaPadrao`.

Claim	Origem	Uso
sub/nameidentifier	IdUsuario	Identifica usuario atual.
email	Email	Identificacao e auditoria.
role	DescricaoPerfil	Autorizacao por roles.
id_perfil	IdPerfil	Contexto adicional de perfil.

Migrations

Migration	Responsabilidade
20260521160031_InitialSqlServer	Cria Professores e Alunos, FK Alunos.ProfessorId e seed inicial.
20260521194537_SeedFiftyRecords	Insere base ampla de exemplo para professores e alunos.
20260521202457_AddJwtAuthUsuariosDiretoria	Cria Perfil, Usuario, Diretoria, IdUsuario em Alunos/Professores, indices, seeds de perfis e 20 usuarios.

A API aplica migrations automaticamente no startup. Para SQLite em desenvolvimento, usa `EnsureCreated`.

FluentValidation

Validators sao registrados no pipeline MVC. Eles impedem payloads invalidos antes que os services executem regras de aplicacao.

- `AlunoCreateEditViewModelValidator` : nome, sobrenome, data, professor e usuario.
- `ProfessorCreateEditViewModelValidator` : nome e usuario vinculado.
- `DiretoriaCreateEditViewModelValidator` : nome e usuario vinculado.
- `UsuarioCreateViewModelValidator` : nome, email, telefone e perfil.
- `LoginRequestViewModelValidator` : email e senha.
- `AlterarSenhaViewModelValidator` : senha atual, nova senha forte e confirmacao.

Logging

O projeto possui provider proprio de log em arquivo: `DailyFileLoggerProvider`. Ele grava arquivos diarios na pasta `Backend_API/logs`.

Origem	Evento registrado
Startup	Ambiente, diretorio de logs e preparacao do banco.
Middleware	Metodo, rota, usuario, status HTTP e tempo de execucao.
Controllers	Excecoes capturadas em CRUDs e autenticacao.
AuthController	Login recusado, login aceito e alteracao de senha.

O arquivo `Backend_API/logs/.gitkeep` versiona a pasta. Logs reais sao ignorados pelo Git.

Swagger/OpenAPI

Swagger e configurado em `Program.cs` com comentarios XML, titulo, versao, descricao e esquema de seguranca Bearer.

Fluxo recomendado: executar `POST /api/Auth/login`, copiar o token, clicar em Authorize e informar `Bearer {token}`.

CRUD Completo

Entidade	Rotas	Autorizacao
Aluno	GET /api/Aluno, GET /api/Aluno/{id}, GET /api/Aluno/ByProfessor/{id}, POST, PUT, DELETE	Leitura autenticada; escrita Administrador/Contribuinte; exclusao Administrador.
Professor	GET /api/Professor, GET /api/Professor/{id}, POST, PUT, DELETE	Leitura autenticada; escrita Administrador/Contribuinte; exclusao Administrador.
Diretoria	GET /api/Diretoria, GET /api/Diretoria/{id}, POST, PUT, DELETE	Leitura autenticada; escrita Administrador/Contribuinte; exclusao Administrador.
Usuario	GET /api/usuarios, GET /api/usuarios/{id}, GET /api/usuarios/perfis, POST, PUT, DELETE	Consulta Administrador/Contribuinte; perfis e escrita apenas Administrador.
Auth	POST /api/Auth/login, GET /api/Auth/me, POST /api/Auth/alterar-senha, GET /api/Auth/autorizar	Login publico; demais autenticados ou Administrador.

Testes Unitarios

Os testes ficam em `Backend_API/form_API.Tests` e usam xUnit, Moq, FluentValidation.TestHelper e SQLite em memoria.

- Controllers: validam respostas principais de Aluno e Usuario.
- Services: validam criacao, hash de senha, login e troca de senha.
- Validators: validam regras de entrada.
- Seguranca: teste impede uso de APIs de SQL bruto.

Comando: `dotnet test form_API.Tests/form_API.Tests.csproj` .

Execucao Local

Acao	Comando
Rodar API	<code>cd Backend_API && dotnet run</code>
Testar	<code>dotnet test form_API.Tests/form_API.Tests.csproj</code>
Docker Compose	<code>docker compose -f docker/docker-compose.yml up --build</code>
Swagger	<code>http://localhost:5000/swagger</code> no compose ou URL exibida pelo dotnet run.